

Conectividad

Componentes conexos, fuertemente conexos y Kosaraju

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre de 2026

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

Objetivos de la sesión

Al terminar la clase, usted podrá

- Decir cuándo un grafo no dirigido es conexo y qué es un componente conexo.
- Explicar por qué el alcance parte los vértices de un grafo no dirigido y por qué en un dirigido no lo hace.
- Distinguir conexo débil de conexo fuerte, y decir en qué tipo de grafo tiene sentido cada concepto.
- Reconocer un punto de articulación y un puente, y decir qué se rompe al quitarlos.

Objetivos de la sesión

Y también

- Hallar los componentes conexos con la búsqueda en profundidad y con la de amplitud, en $\Theta(V + E)$.
- Explicar qué son G^T y G^{SCC} , y por qué el grafo de componentes nunca tiene ciclos.
- Ejecutar el algoritmo de Kosaraju a mano e implementarlo en $\Theta(V + E)$.

La pregunta que ordena la sesión

Una sola pregunta, en dos versiones

Dados dos vértices, ¿se puede ir de uno al otro? Con aristas sin dirección la respuesta es sí o no, y agrupa los vértices en bloques. Con aristas dirigidas hay que preguntar dos veces, de ida y de vuelta, y las dos respuestas no tienen por qué coincidir.

Lo que se construye hoy

Los bloques del caso no dirigido salen de un recorrido repetido y cuestan $\Theta(V + E)$. Los del caso dirigido cuestan lo mismo, pero hacen falta dos recorridos y el grafo con las aristas al revés.

Contenido

- 1 Antes de empezar
- 2 Recuento**
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

El bosque de la búsqueda en profundidad

Los predecesores

La búsqueda en profundidad guarda en $v.\pi$ el vértice desde el cual se descubrió v . Las aristas $(v.\pi, v)$ forman el *bosque de profundidad*: un árbol por cada vértice desde el que hubo que arrancar de nuevo. Es bosque y no árbol porque un solo arranque no tiene por qué alcanzar todo el grafo.

Los tres colores

- **Blanco**: no ha sido descubierto.
- **Gris**: está en proceso, o sea en la pila de recursión.
- **Negro**: ya se exploró todo lo que sale de él.

Dos marcas de tiempo por vértice

Definición (CLRS, Sección 22.3)

$v.d$ es el instante en que la búsqueda descubre v y lo pinta de gris. $v.f$ es el instante en que termina de explorar todos sus adyacentes y lo pinta de negro. Un vértice está gris exactamente durante el intervalo $[v.d, v.f]$.

El reloj

Cada descubrimiento y cada finalización avanzan el reloj en uno, y hay $|V|$ de cada clase, así que los tiempos son los enteros de 1 a $2|V|$, todos distintos. Siempre $v.d < v.f$.

Los dos tiempos, en código

```
def en_blanco(grafo):  
    # Los tres diccionarios del recorrido, uno por vertice.  
    color = {}  
    d = {}  
    f = {}  
    for v in grafo:  
        color[v] = BLANCO  
        d[v] = 0  
        f[v] = 0  
    resultado = (color, d, f)  
    return resultado
```

Los dos tiempos, en código

```
def tiempos(grafo):  
    # Los tiempos de descubrimiento y de finalizacion.  
    color, d, f = en_blanco(grafo)  
    reloj = [0]  
  
    def visit(u):  
        color[u] = GRIS  
        reloj[0] = reloj[0] + 1  
        d[u] = reloj[0]  
        for v in grafo[u]:  
            if color[v] == BLANCO:  
                visit(v)  
        color[u] = NEGRO  
        reloj[0] = reloj[0] + 1  
        f[u] = reloj[0]  
  
    for u in grafo:  
        if color[u] == BLANCO:  
            visit(u)  
    resultado = (d, f)  
    return resultado
```

Los dos tiempos, en código

Sobre el grafo dirigido de la sesión

Devuelve a : $1/16$, b : $2/15$, c : $3/12$, d : $4/7$, e : $13/14$, f : $9/10$, g : $8/11$, h : $5/6$. El reloj va en una lista de un elemento para que las llamadas anidadas compartan el mismo contador.

El teorema del paréntesis

Teorema (Teorema 22.7 de CLRS)

Para cualquier par de vértices u y v de un recorrido en profundidad se cumple exactamente una de estas tres condiciones: los intervalos $(u.d, u.f)$ y $(v.d, v.f)$ son disjuntos, y ninguno de los dos es descendiente del otro en el bosque de profundidad; $(u.d, u.f) \subseteq (v.d, v.f)$, y u es descendiente de v ; o $(v.d, v.f) \subseteq (u.d, u.f)$, y v es descendiente de u .

El anidamiento

Escribiendo un paréntesis que abre en cada $v.d$ y uno que cierra en cada $v.f$ queda una expresión bien formada, del estilo $(u (v (x (y (z))))$ a): los descendientes de v son x , y y z ; los de u son todos los demás. Los paréntesis cruzados, como $(10, 20)$ contra $(15, 30)$, no aparecen nunca.

Los cuatro tipos de arista

Clasificación de (u, v) según el color de v

Tipo	Color de v	Qué significa
Árbol	blanco	v se descubre por esa arista
Retroceso	gris	v es ancestro de u
Avance	negro	v es descendiente no directo de u
Cruzada	negro	sin relación de ancestro entre ellos

Para separar avance de cruzada

Con v negro, la arista es de avance si $u.d < v.d$ y cruzada si $v.d < u.d$.
En un grafo no dirigido solo salen aristas de árbol y de retroceso.

Los cuatro tipos de arista

Una arista de retroceso es un ciclo

Si al mirar (u, v) el vértice v está gris, hay un camino de v hasta u por el árbol y la arista (u, v) lo cierra.

De todo eso, lo que se usa hoy

Los tiempos de finalización

De los dos tiempos, hoy solo pesa $v.f$. El orden de finalización leído al revés es lo que le dice al algoritmo de Kosaraju por dónde empezar, y el teorema del paréntesis es lo que garantiza que ese orden significa algo.

El bosque

Que la búsqueda deje un bosque y no un solo árbol es justo lo que hace falta para contar componentes: cada arranque nuevo es un bloque de vértices que no se alcanzaba desde los anteriores.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad**
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

Camino y alcance

Definición (Paseo, camino, ciclo)

Un paseo es una secuencia de vértices conectados por aristas, donde los vértices se pueden repetir. Un camino es un paseo que no repite vértices. Un ciclo es un camino cuyo vértice inicial es igual al final.

Definición (Alcanzabilidad)

v es alcanzable desde u si existe un camino de u a v .

Distancia

La distancia entre u y v es el número de aristas del camino más corto que los conecta. Si no existe tal camino, la distancia es ∞ , que es otra forma de decir que v no es alcanzable desde u .

En un grafo no dirigido el alcance es una relación de equivalencia

Teorema

En un grafo no dirigido, la relación “ v es alcanzable desde u ” cumple las tres propiedades de una relación de equivalencia:

- **Reflexiva:** *todo vértice es alcanzable desde sí mismo, por el camino de longitud 0.*
- **Simétrica:** *si v es alcanzable desde u , entonces u es alcanzable desde v .*
- **Transitiva:** *si w es alcanzable desde u y v es alcanzable desde w , entonces v es alcanzable desde u .*

En un grafo no dirigido el alcance es una relación de equivalencia

Por qué importa

Una relación de equivalencia parte el conjunto sobre el que actúa en clases disjuntas que lo cubren entero. Esas clases son los componentes conexos: cada vértice cae en uno y en uno solo, sin que haga falta demostrarlo aparte.

La simetría es la que se cae

En un grafo dirigido las otras dos siguen valiendo, pero la simetría no: que se pueda ir de u a v no dice nada sobre el viaje de vuelta. El alcance deja de partir los vértices y toca cambiar de herramienta.

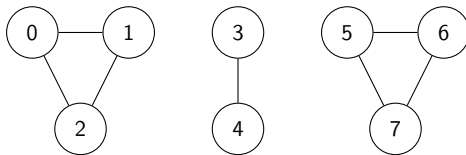
Grafo conexo

Definición (Grafo conexo)

Un grafo no dirigido $G(V, E)$ es conexo si para todo par de vértices $u, v \in V$, existe un camino entre ellos.

Una condición necesaria, no suficiente

Para que un grafo no dirigido sea conexo debe cumplir $|E| \geq |V| - 1$: con menos aristas no alcanza para amarrar todos los vértices. La condición no basta, y el grafo de abajo lo muestra: tiene ocho vértices y siete aristas, y aun así no es conexo.



Componente conexo

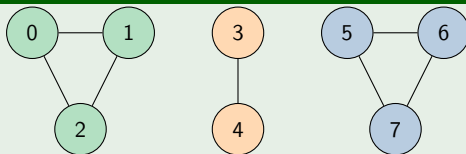
Definición (Componente conexo)

Subgrafo conexo maximal de un grafo no dirigido. Es decir, un subconjunto de vértices donde

- 1 existe un camino entre cualquier par de vértices del subconjunto, y*
- 2 no se puede agregar ningún vértice adicional del grafo sin perder esa propiedad.*

Componente conexo

Tres componentes



$\{0, 1, 2\}$, $\{3, 4\}$ y $\{5, 6, 7\}$. El conjunto $\{0, 1\}$ no es un componente: cumple la primera condición pero no la segunda, porque cabe el 2.

Lo que los componentes garantizan

Cuatro propiedades

- 1 **Intersección vacía:** dos componentes distintos no comparten ningún vértice.
- 2 **Unión completa:** la unión de todos los componentes es $V(G)$.
- 3 **Maximalidad:** cada componente es un subgrafo conexo maximal.
- 4 G es conexo si y solo si tiene exactamente un componente.

La consecuencia que usa el algoritmo

No existe ninguna arista entre vértices de componentes distintos. Si la hubiera, los dos componentes estarían unidos por un camino y serían uno solo, en contra de la maximalidad. Por eso un recorrido que arranca dentro de un componente no puede salirse de él.

En grafos dirigidos: débil y fuerte

Definición (Conectividad fuerte)

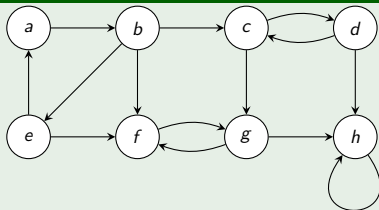
Un grafo dirigido es fuertemente conexo si para todo par de vértices u, v existe un camino dirigido de u a v y de v a u .

Definición (Conectividad débil)

Un grafo dirigido es débilmente conexo si al ignorar la dirección de las aristas, el grafo no dirigido subyacente es conexo.

En grafos dirigidos: débil y fuerte

Un grafo débilmente conexo que no es fuertemente conexo



Ignorando las flechas se llega de cualquier vértice a cualquier otro. Con las flechas puestas, de *h* no se sale.

En qué grafo tiene sentido cada concepto

Concepto	No dirigido	Dirigido
Grafo conexo	sí	no
Componente conexo	sí	no
Punto de articulación	sí	no
Puente	sí	no
Conexo débil	no	sí
Conexo fuerte	no	sí
Componente fuertemente conexo	no	sí

En qué grafo tiene sentido cada concepto

Cruzar las columnas no compila

Preguntar por los componentes conexos de un grafo dirigido no está definido: hay que decir si se ignoran las direcciones, y entonces la pregunta es por los débiles, o si no se ignoran, y entonces es por los fuertes. Los dos cálculos dan cosas distintas.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos**
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

Lo que alcanza un recorrido

Teorema (Teorema 22.5 de CLRS)

Si se ejecuta la búsqueda en amplitud o la búsqueda en profundidad desde un vértice origen s en un grafo no dirigido, entonces

- *todos los vértices v para los cuales existe un camino entre s y v son visitados por el algoritmo, y*
- *el conjunto de vértices visitados corresponde exactamente al componente conexo que contiene a s .*

El algoritmo ya está escrito

Un recorrido entrega un componente entero, ni más ni menos. Faltan los vértices que ese recorrido no tocó, y esos son los de los otros componentes: se arranca otro recorrido desde el primero que quede sin visitar y se repite hasta que no quede ninguno.

Con la búsqueda en profundidad

ComponentesConexosDFS(G)

1. Para cada nodo v :
 - 1.1 Si v no ha sido visitado crear un nuevo componente y ejecutar $CCDFSAux(v)$

$CCDFSAux(v)$

1. Agregar v al componente actual
2. Marcar v como visitado
3. Para cada nodo u adyacente a v :
 - 3.1 Si u no ha sido visitado ejecutar $CCDFSAux(u)$

Con la búsqueda en profundidad

El orden de los pasos 1 y 2

Marcar y agregar ocurren antes de mirar a los adyacentes. Si se marcara después, un ciclo haría que el recorrido volviera a entrar al mismo vértice y la recursión no terminaría.

Con la búsqueda en amplitud

ComponentesConexosBFS(G)

1. Para cada nodo v :
 - 1.1 Si v no ha sido visitado crear un nuevo componente y ejecutar $CCBFSAux(v)$

Con la búsqueda en amplitud

$CCBFSA_{ux}(v)$

1. Agregar v a la cola P
2. Marcar v como visitado
3. Agregar v al componente actual
4. Mientras la cola P no este vacia:
 - 4.1 Obtener el tope w de la cola P
 - 4.2 Para cada nodo u adyacente a w :
 - 4.3 Si u no ha sido visitado:
 - 4.3.1 Agregar u a la cola P
 - 4.3.2 Marcar u como visitado
 - 4.3.3 Agregar u al componente actual

Con la búsqueda en amplitud

La única diferencia

Dónde espera lo pendiente. En profundidad, en la pila; en amplitud, en la cola. El reparto de vértices en componentes sale igual; lo que cambia es el orden en que se visitan dentro de cada uno.

El grafo

```
if __name__ == "__main__":  
    # Grafo no dirigido de ocho vertices con tres componentes.  
    grafo = {  
        0: [1, 2],  
        1: [0, 2],  
        2: [0, 1],  
        3: [4],  
        4: [3],  
        5: [6, 7],  
        6: [5, 7],  
        7: [5, 6],  
    }
```

El grafo

Un diccionario de adyacencia

La clave es el vértice y el valor es la lista de sus vecinos. En un grafo no dirigido cada arista aparece dos veces, una en cada extremo: la arista 0–1 está en `grafo[0]` y en `grafo[1]`. Con las claves 0 a $n - 1$, las marcas de visitado caben en una lista indexada por vértice.

La profundidad recursiva

```
def cc_dfs_aux(grafo, v, visitado, actual):  
    # Marca v, lo agrega al componente y baja por sus vecinos.  
    actual.append(v)  
    visitado[v] = True  
    for u in grafo[v]:  
        if not visitado[u]:  
            cc_dfs_aux(grafo, u, visitado, actual)
```

La profundidad recursiva

```
def componentes_conexos(grafo):  
    # grafo: dict que asocia cada vertice a la lista de vecinos.  
    # Devuelve los componentes, cada uno como lista de vertices.  
    n = len(grafo)  
    visitado = [False] * n  
    componentes = []  
    for v in grafo:  
        if not visitado[v]:  
            actual = []  
            cc_dfs_aux(grafo, v, visitado, actual)  
            componentes.append(actual)  
    return componentes
```

La profundidad recursiva

Lo que devuelve

Una lista de listas: `[[0, 1, 2], [3, 4], [5, 6, 7]]`. El paso 1 del algoritmo es el `for` de `componentes_conexos`, y “crear un nuevo componente” es la lista actual que se pasa a la recursión y se guarda al volver.

La profundidad con pila explícita

```
def cc_dfs_aux_con_pila(grafo, v, visitado, actual):  
    # Lo pendiente queda en una lista, no en la pila de llamadas.  
    pila = [v]  
    visitado[v] = True  
    while len(pila) > 0:  
        w = pila.pop()  
        actual.append(w)  
        for u in grafo[w]:  
            if not visitado[u]:  
                visitado[u] = True  
                pila.append(u)
```

La profundidad con pila explícita

```
def componentes_conexos_con_pila(grafo):  
    # Mismo reparto de vertices que la version recursiva.  
    n = len(grafo)  
    visitado = [False] * n  
    componentes = []  
    for v in grafo:  
        if not visitado[v]:  
            actual = []  
            cc_dfs_aux_con_pila(grafo, v, visitado, actual)  
            componentes.append(actual)  
    return componentes
```

La profundidad con pila explícita

Qué cambió

Lo pendiente pasó de la pila de llamadas a la lista `pila`. El reparto de vértices es el mismo; dentro de cada componente el orden de visita cambia, porque la pila entrega el último vecino que entró: el primer componente sale `[0, 2, 1]`. Con un grafo de 10^5 vértices en fila la versión recursiva pasa del límite de recursión de Python y esta no.

La amplitud

```
def cc_bfs_aux(grafo, v, visitado, actual):  
    # La cola es una lista con un puntero de cabeza que avanza.  
    cola = [v]  
    cabeza = 0  
    visitado[v] = True  
    actual.append(v)  
    while cabeza < len(cola):  
        w = cola[cabeza]  
        cabeza = cabeza + 1  
        for u in grafo[w]:  
            if not visitado[u]:  
                cola.append(u)  
                visitado[u] = True  
                actual.append(u)
```

La amplitud

```
def componentes_conexos_bfs(grafo):  
    n = len(grafo)  
    visitado = [False] * n  
    componentes = []  
    for v in grafo:  
        if not visitado[v]:  
            actual = []  
            cc_bfs_aux(grafo, v, visitado, actual)  
            componentes.append(actual)  
    return componentes
```

La amplitud

Las tres versiones sobre el grafo de ocho vértices

El mismo reparto en las tres: $[0, 1, 2]$, $[3, 4]$, $[5, 6, 7]$, salvo el orden interno de la versión con pila. La cola es una lista con un puntero de cabeza que solo avanza; no hace falta sacar elementos del frente.

Lo que cuesta

Teorema

ComponentesConexosDFS sobre un grafo no dirigido $G(V, E)$ representado con listas de adyacencia corre en $\Theta(V + E)$.

Demostración

Se procede de forma directa, contando por separado el ciclo del paso 1 y las llamadas a *CCDFSAux*.

Lo que cuesta

Desarrollo

- El ciclo del paso 1 consulta la marca de cada vértice una vez y nada más: $\Theta(V)$ sin contar lo que hagan las llamadas.
- $CCDFS_{Aux}(v)$ solo se invoca con v sin visitar, y lo primero que hace es marcarlo. Un vértice se marca una sola vez en toda la ejecución, así que hay exactamente $|V|$ llamadas.
- Cada llamada recorre la lista de adyacencia de su vértice, que tiene $\deg(v)$ entradas. Sumando sobre todas, $\sum_{v \in V} \deg(v) = 2|E|$, porque cada arista no dirigida aparece en dos listas.
- En total, $\Theta(V) + \Theta(V) + \Theta(E)$.
- La cota inferior es la misma: el algoritmo lee entera la lista de adyacencia de cada vértice, o sea $|V| + 2|E|$ casillas, y no puede hacer menos.

Lo que cuesta

Conclusión

Por lo tanto, se puede concluir que *ComponentesConexosDFS* corre en $\Theta(V + E)$. La versión con amplitud tiene la misma cuenta: cada vértice entra a la cola una sola vez y sale una sola vez, y el recorrido de las listas de adyacencia es el mismo.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes**
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

Qué se rompe al quitar una pieza

La pregunta

Un grafo conexo tiene un componente. Al borrar un vértice o una arista puede seguir teniendo uno, o puede partirse. Los vértices y las aristas que lo parten son los que sostienen la conexión, y son los que hay que vigilar.

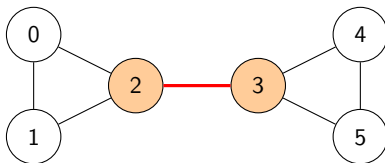
Definición (Punto de articulación)

Sea un grafo $G = (V, E)$. Un vértice $v \in V$ es denominado un punto de articulación o vértice de corte si al remover v y todas las aristas asociadas a v se incrementa el número de componentes conexos en G .

Definición (Puente)

Sea un grafo $G = (V, E)$. Una arista $(u, v) \in E$ es denominada un puente, istmo o arista de corte si al remover (u, v) se incrementa el número de componentes conexos en G .

Dos triángulos unidos por una arista



Lo que se ve

El grafo es conexo: un componente. Marcados quedan los dos puntos de articulación, 2 y 3, y el único puente, la arista (2,3).

Dos triángulos unidos por una arista

Probando pieza por pieza

- Sin el vértice 2 quedan $\{0, 1\}$ y $\{3, 4, 5\}$: dos componentes, luego el 2 es punto de articulación. Por simetría, el 3 también.
- Sin la arista $(2, 3)$ quedan $\{0, 1, 2\}$ y $\{3, 4, 5\}$: dos componentes, luego $(2, 3)$ es un puente.
- Sin el vértice 0, o sin la arista $(0, 1)$, el grafo sigue conexo: los demás vértices del triángulo se alcanzan dando la vuelta por el otro lado.

Dos observaciones sobre los puentes

Un puente no está en ningún ciclo

Una arista (u, v) es un puente cuando no hace parte de un ciclo. Si estuviera en uno, al borrarla quedaría el resto del ciclo como camino alternativo de u a v , y no se partiría nada.

Cuántos puede haber

Un grafo con n nodos puede tener como máximo $n - 1$ puentes: los puentes de un grafo conexo son exactamente las aristas del árbol que queda al contraer cada pieza sin puentes a un solo vértice, y un árbol de n vértices tiene $n - 1$ aristas.

Dos observaciones sobre los puentes

Los dos conceptos no van de la mano

Un extremo de un puente casi siempre es punto de articulación, pero hay puntos de articulación sin ningún puente al lado: dos triángulos pegados por un vértice comparten ese vértice, que es punto de articulación, y el grafo no tiene puentes.

Dónde importan

Puntos únicos de falla

Un enrutador que es punto de articulación deja incomunicadas dos zonas cuando se cae, y un enlace que es puente hace lo mismo. En una red eléctrica o vial, el puente es el tramo sin ruta alterna: un mantenimiento o un derrumbe parte el mapa en dos. En una red social, el contacto único entre dos comunidades que por lo demás no se hablan.

Grafos 2-conexos

Un grafo es *2-conexo* si no tiene puntos de articulación: entre cualquier par de vértices hay al menos dos caminos que no comparten vértices intermedios, y por eso ninguna caída deja incomunicado a nadie. Es lo que se le pide a una red que tiene que aguantar una falla.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos**
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

La definición

Definición (Componente fuertemente conexo)

Un componente fuertemente conexo (strongly connected component, SCC) de un grafo dirigido es un subconjunto de vértices tal que, para todo par de vértices u y v dentro del componente, existe un camino dirigido de u a v y también de v a u .

Maximal

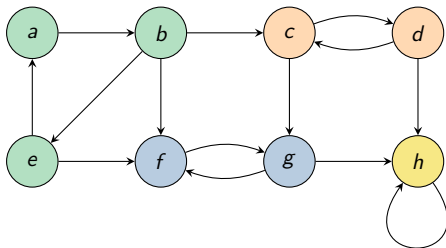
Un componente fuertemente conexo es maximal: si se agrega cualquier vértice adicional, se pierde la propiedad de alcanzabilidad mutua entre todos los vértices.

La definición

Por qué la maximalidad no sobra

Sin ella, todo vértice suelto sería un componente válido y la respuesta no sería única. Con ella, la alcanzabilidad mutua vuelve a ser relación de equivalencia —ahora sí simétrica, porque se exigen los dos caminos— y los componentes vuelven a partir a V .

Cuatro componentes en el grafo de la sesión



Cuatro componentes en el grafo de la sesión

Los cuatro

$\{a, b, e\}$, $\{c, d\}$, $\{f, g\}$ y $\{h\}$.

- En $\{a, b, e\}$: $a \rightarrow b \rightarrow e \rightarrow a$ cierra el ciclo, así que los tres se alcanzan entre sí.
- $\{c, d\}$ vive de las dos aristas $c \rightarrow d$ y $d \rightarrow c$.
- $\{f, g\}$, igual, con $f \rightarrow g$ y $g \rightarrow f$.
- h está solo: llegan aristas a él, pero la única que sale es $h \rightarrow h$, que no lleva a ninguna parte.

Un vértice siempre es alcanzable desde sí mismo

Por el camino de longitud 0. Por eso h es un componente, y no un vértice que se quedó por fuera. Todo vértice pertenece a algún componente, y los tamaños de todos los componentes suman $|V|$.

El grafo transpuesto

Definición (Grafo transpuesto G^T)

Es el grafo obtenido al invertir la dirección de todas las aristas de G . Se cumple que G^T tiene los mismos componentes fuertemente conexos que G , solo que las aristas están en dirección contraria.

Demostración de que los componentes se conservan

Se procede de forma directa, mostrando que la alcanzabilidad mutua es la misma relación en los dos grafos.

El grafo transpuesto

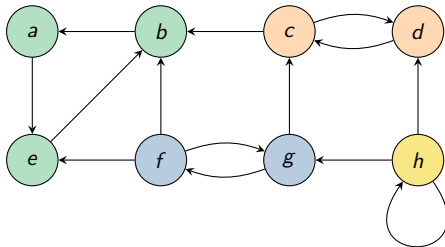
Desarrollo

Un camino $u \rightarrow x_1 \rightarrow \cdots \rightarrow x_k \rightarrow v$ en G es, leído al revés, el camino $v \rightarrow x_k \rightarrow \cdots \rightarrow x_1 \rightarrow u$ en G^T , porque cada arista del primero está invertida en el segundo. Así que hay camino de u a v en G si y solo si hay camino de v a u en G^T . Pedir los dos caminos, de ida y de vuelta, es entonces pedir exactamente lo mismo en los dos grafos, y los subconjuntos maximales que lo cumplen coinciden.

Conclusión

Por lo tanto, se puede concluir que G y G^T tienen los mismos componentes fuertemente conexos.

El transpuesto del grafo de la sesión



Los mismos colores

Las flechas cambiaron de sentido y los bloques quedaron intactos. Lo que sí cambió es por dónde se puede salir de cada bloque: en G de $\{a, b, e\}$ se llega a todos los demás; en G^T , a ninguno.

El transpuesto del grafo de la sesión

Construirlo cuesta $\Theta(V + E)$

Se recorre cada lista de adyacencia una vez y se agrega la arista al revés en la lista del otro extremo.

El grafo de componentes

Definición (Grafo de componentes G^{SCC})

Sea G^{SCC} el grafo donde cada nodo representa un componente fuertemente conexo y existe una arista entre dos componentes si hay al menos una arista en G desde algún vértice del primer componente hacia algún vértice del segundo.

Teorema

G^{SCC} es siempre un DAG, es decir, no contiene ciclos.

Demostración

Se procede por contradicción.

El grafo de componentes

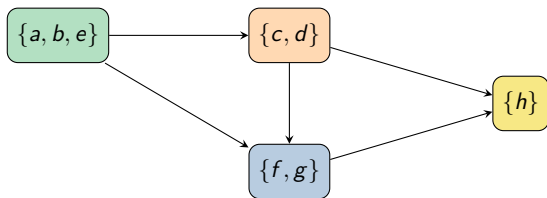
Desarrollo

Suponga que G^{SCC} tiene un ciclo $C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_k \rightarrow C_1$ con $k \geq 2$ componentes distintos. Tome $u \in C_i$ y $v \in C_j$ cualesquiera, con $i \neq j$. Recorriendo el ciclo hacia adelante desde C_i se llega a C_j , y cada paso $C_r \rightarrow C_{r+1}$ es una arista de G entre algún vértice de C_r y alguno de C_{r+1} , que se completa a camino porque dentro de cada componente todo vértice alcanza a todo vértice. Hay entonces camino de u a v . Siguiendo la vuelta completa del ciclo se llega de v a u por el mismo argumento. Los vértices de $C_1 \cup \dots \cup C_k$ se alcanzan mutuamente, así que ninguno de esos C_i era maximal, en contra de la definición de componente fuertemente conexo.

Conclusión

Por lo tanto, se puede concluir que G^{SCC} no tiene ciclos y es un DAG. De ahí que admita un orden topológico.

G^{SCC} del grafo de la sesión



G^{SCC} del grafo de la sesión

De dónde sale cada arista

- $\{a, b, e\} \rightarrow \{c, d\}$ por $b \rightarrow c$.
- $\{a, b, e\} \rightarrow \{f, g\}$ por $b \rightarrow f$ y por $e \rightarrow f$: dos aristas de G , una sola de G^{SCC} .
- $\{c, d\} \rightarrow \{f, g\}$ por $c \rightarrow g$.
- $\{c, d\} \rightarrow \{h\}$ por $d \rightarrow h$.
- $\{f, g\} \rightarrow \{h\}$ por $g \rightarrow h$.

Las aristas de G que quedan dentro de un componente, como $a \rightarrow b$ o $h \rightarrow h$, no aparecen: serían lazos.

Los tiempos de un componente

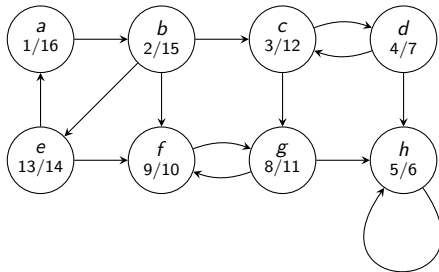
Definición ($d(C)$ y $f(C)$)

Para un componente C ,

$$d(C) = \min_{v \in C}(v.d) \quad y \quad f(C) = \max_{v \in C}(v.f).$$

El primer vértice del componente que la búsqueda descubre y el último que termina.

Los tiempos de un componente



Los tiempos de un componente

Los cuatro componentes con sus tiempos

Componente	Tiempos de sus vértices	$d(C)$	$f(C)$
$\{a, b, e\}$	1/16, 2/15, 13/14	1	16
$\{c, d\}$	3/12, 4/7	3	12
$\{f, g\}$	9/10, 8/11	8	11
$\{h\}$	5/6	5	6

La propiedad en la que se apoya Kosaraju

Teorema (CLRS, Sección 22.5)

Si existe una arista de C a C' en G^{SCC} , entonces $f(C) > f(C')$ en una búsqueda en profundidad sobre G .

Qué dice en palabras

En el grafo de componentes, las flechas apuntan siempre de mayor a menor tiempo de finalización. El componente con el f más grande de todos no tiene ninguna arista que le entre: es una fuente de G^{SCC} .

La propiedad en la que se apoya Kosaraju

Sobre las cinco aristas del ejemplo

Arista de G^{SCC}	$f(C)$	$f(C')$
$\{a, b, e\} \rightarrow \{c, d\}$	16	12
$\{a, b, e\} \rightarrow \{f, g\}$	16	11
$\{c, d\} \rightarrow \{f, g\}$	12	11
$\{c, d\} \rightarrow \{h\}$	12	6
$\{f, g\} \rightarrow \{h\}$	11	6

La propiedad en la que se apoya Kosaraju

Con $d(C)$ no funciona

En la misma tabla, $d(\{h\}) = 5$ y $d(\{f, g\}) = 8$: el componente $\{h\}$ se descubre antes que $\{f, g\}$ aunque la arista vaya de $\{f, g\}$ a $\{h\}$. Ordenar por tiempo de descubrimiento no sirve; solo el de finalización respeta las flechas.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju**
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

La idea

Dos recorridos y un grafo al revés

El componente con el f más grande es una fuente de G^{SCC} : nada le entra. En G^T eso se voltea y nada le sale. Un recorrido que arranque ahí sobre G^T toca todo el componente, porque por dentro sigue fuertemente conexo, y no puede irse a otro, porque no hay por dónde.

Y después

Ese componente se marca y se sigue con el vértice sin asignar que tenga el f más grande de los que quedan. Sus aristas de salida en G^T solo pueden ir a componentes ya marcados, y el recorrido los ignora.

Atribución

El algoritmo se le atribuye a Sambasiva Rao Kosaraju (1978), que no lo publicó. Corre en $O(|V| + |E|)$ con listas de adyacencia.

El algoritmo

Kosaraju(G)

1. Hacer DFS sobre G y obtener orden topologico ord
2. Obtener el grafo transpuesto G'
3. Para cada nodo v de acuerdo al orden en ord ejecutar $Asignar(v, v)$

Asignar(v, g)

1. Si v no ha sido asignado a ningun componente:
2. Asignar v al componente asociado a g
3. Para cada nodo u adyacente a v en G' ejecutar $Asignar(u, g)$

El algoritmo

Qué es *ord*

Los vértices por f decreciente, con el mismo cálculo del orden topológico: se agrega cada vértice cuando la profundidad termina con él y al final se invierte la lista. *ord* es un orden topológico de G^{SCC} , que es el grafo sobre el que el algoritmo lo usa: recorrerlo de izquierda a derecha es visitar los componentes desde las fuentes hacia los sumideros.

Qué es *g*

El vértice que arrancó la asignación, y que hace de nombre del componente. Dos vértices quedan en el mismo componente si les tocó el mismo *g*.

Por qué funciona

Teorema

Kosaraju(G) asigna el mismo valor de g a dos vértices si y solo si están en el mismo componente fuertemente conexo de G .

Demostración

Se procede por inducción sobre el número de llamadas del paso 3 que encuentran su vértice sin asignar, apoyándose en que si hay arista de C a C' en G^{SCC} entonces $f(C) > f(C')$.

Por qué funciona

Desarrollo: la primera llamada

- El primer v de ord es el de f máximo, y su componente C cumple $f(C) = v.f$, el máximo global. Ningún C' puede tener arista hacia C , porque eso exigiría $f(C') > f(C)$. Sin aristas entrantes en G^{SCC} , en G^T el componente C no tiene aristas salientes hacia otro componente. *Asignar*(v, v) recorre entonces C completo —es fuertemente conexo, también en G^T — y no sale de él.

Por qué funciona

Desarrollo: el paso inductivo

- Suponga que las primeras k llamadas asignaron exactamente k componentes completos. La siguiente llamada que encuentra su vértice sin asignar cae en un componente C que tiene el f más grande entre los que quedan. Si en G^{SCC} hubiera una arista $C' \rightarrow C$ con C' sin asignar, sería $f(C') > f(C)$, en contra de que C es el máximo de los que quedan. Así que toda arista que entra a C viene de un componente ya asignado, y en G^T toda arista que sale de C va a un componente ya asignado. La recursión se detiene en ellos por el paso 1 de *Asignar*, y vuelve a cubrir C completo y nada más.

Por qué funciona

Conclusión

Por lo tanto, se puede concluir que cada llamada del paso 3 que encuentra su vértice sin asignar pinta un componente fuertemente conexo entero, y que al terminar *ord* todos los vértices quedaron repartidos en sus componentes.

El orden por finalización

```
def orden_aux(grafo, v, visitado, orden):  
    # v se agrega al terminar con el: orden queda por f creciente.  
    visitado[v] = True  
    for u in grafo[v]:  
        if not visitado[u]:  
            orden_aux(grafo, u, visitado, orden)  
    orden.append(v)
```

El orden por finalización

```
def orden_por_finalizacion(grafo):  
    # Los vertices por tiempo de finalizacion decreciente.  
    visitado = {}  
    for v in grafo:  
        visitado[v] = False  
    orden = []  
    for v in grafo:  
        if not visitado[v]:  
            orden_aux(grafo, v, visitado, orden)  
    orden.reverse()  
    return orden
```

El orden por finalización

Sin reloj

No hace falta calcular $v.f$: basta agregar el vértice justo cuando la recursión vuelve de él. La lista queda por f creciente y se invierte al final. Sobre el grafo de la sesión devuelve ['a', 'b', 'e', 'c', 'g', 'f', 'd', 'h'].

El orden por finalización, con pila explícita

```
def orden_aux_con_pila(grafo, v, visitado, orden):  
    # Cada vertice entra dos veces: la primera lo descubre y  
    # la segunda lo finaliza, que es cuando se agrega a orden.  
    pila = [(v, False)]  
    while len(pila) > 0:  
        w, finalizando = pila.pop()  
        if finalizando:  
            orden.append(w)  
        elif not visitado[w]:  
            visitado[w] = True  
            pila.append((w, True))  
            for u in grafo[w]:  
                if not visitado[u]:  
                    pila.append((u, False))
```

El orden por finalización, con pila explícita

```
def orden_por_finalizacion_con_pila(grafo):  
    visitado = {}  
    for v in grafo:  
        visitado[v] = False  
    orden = []  
    for v in grafo:  
        if not visitado[v]:  
            orden_aux_con_pila(grafo, v, visitado, orden)  
    orden.reverse()  
    return orden
```

El orden por finalización, con pila explícita

La marca finalizando

La recursiva hace dos cosas con cada vértice en momentos distintos: descubrirlo antes de bajar y agregarlo al volver. Con la pila hay que escribir esos dos momentos aparte, y por eso cada vértice se apila dos veces, primero con False y después con True. La marca True queda debajo de todos sus adyacentes, o sea que sale después que ellos.

El transpuesto y la asignación

```
def grafo_transpuesto(grafo):  
    # Las mismas aristas con la direccion invertida.  
    GT = {}  
    for v in grafo:  
        GT[v] = []  
    for v in grafo:  
        for u in grafo[v]:  
            GT[u].append(v)  
    return GT
```

El transpuesto y la asignación

```
def asignar(GT, v, g, comp):  
    # v y lo que alcance en GT sin asignar quedan con g.  
    if comp[v] is None:  
        comp[v] = g  
        for u in GT[v]:  
            asignar(GT, u, g, comp)
```

El transpuesto y la asignación

```
def asignar_con_pila(GT, v, g, comp):  
    # Lo pendiente queda en una lista, no en la pila de llamadas.  
    pila = [v]  
    while len(pila) > 0:  
        w = pila.pop()  
        if comp[w] is None:  
            comp[w] = g  
            for u in GT[w]:  
                pila.append(u)
```

El transpuesto y la asignación

None hace de marca de visitado

No hace falta un diccionario aparte: un vértice con `comp[w]` distinto de `None` ya tiene componente y la recursión se detiene ahí. Esa única comprobación cubre a la vez el caso del vértice ya visto en este recorrido y el del vértice de un componente anterior.

Kosaraju en Python

```
def kosaraju(grafo):  
    # grafo: dict que asocia cada vertice a la lista de sucesores.  
    # Devuelve la lista de componentes fuertemente conexos.  
    orden = orden_por_finalizacion(grafo)  
    GT = grafo_transpuesto(grafo)  
    comp = {}  
    for v in grafo:  
        comp[v] = None  
    for v in orden:  
        asignar(GT, v, v, comp)  
    componentes = agrupar(grafo, comp, orden)  
    return componentes
```

Kosaraju en Python

```
def kosaraju_con_pila(grafo):  
    orden = orden_por_finalizacion_con_pila(grafo)  
    GT = grafo_transpuesto(grafo)  
    comp = {}  
    for v in grafo:  
        comp[v] = None  
    for v in orden:  
        asignar_con_pila(GT, v, v, comp)  
    componentes = agrupar(grafo, comp, orden)  
    return componentes
```

Kosaraju en Python

```
def agrupar(grafo, comp, orden):  
    # Junta los vertices con el mismo representante, en el  
    # orden en que los componentes fueron apareciendo.  
    grupos = {}  
    for v in orden:  
        if comp[v] not in grupos:  
            grupos[comp[v]] = []  
    for v in grafo:  
        grupos[comp[v]].append(v)  
    componentes = []  
    for g in grupos:  
        componentes.append(grupos[g])  
    return componentes
```

Kosaraju en Python

Sobre el grafo de la sesión

Las dos devuelven [['a', 'b', 'e'], ['c', 'd'], ['f', 'g'], ['h']], y en ese orden, que es el de G^{SCC} topológicamente ordenado.

Lo que cuesta

Teorema

Kosaraju(G) sobre un grafo dirigido $G(V, E)$ con listas de adyacencia corre en $\Theta(V + E)$.

Demostración

Se procede de forma directa, sumando los tres pasos del algoritmo.

Lo que cuesta

Desarrollo

- Paso 1: una búsqueda en profundidad completa sobre G . Marca cada vértice una vez y recorre cada lista de adyacencia una vez: $\Theta(V + E)$.
- Paso 2: construir G^T recorre las $|V|$ listas y agrega cada una de las $|E|$ aristas invertida: $\Theta(V + E)$.
- Paso 3: *Asignar* entra al cuerpo del *if* una sola vez por vértice, porque de ahí en adelante *comp[v]* deja de ser *None*. Las llamadas que se detienen en el paso 1 son a lo sumo una por arista de G^T , y G^T tiene las mismas $|E|$ aristas que G : $\Theta(V + E)$.
- Sumando los tres, $3\Theta(V + E)$, que es $\Theta(V + E)$.

Lo que cuesta

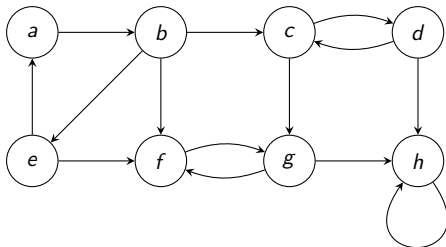
Conclusión

Por lo tanto, se puede concluir que $Kosaraju(G)$ corre en $\Theta(V + E)$.
Recorrer el grafo dos veces y copiarlo una no cambia el orden de crecimiento; cambia la constante.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso**
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias

El grafo de partida



Las catorce aristas

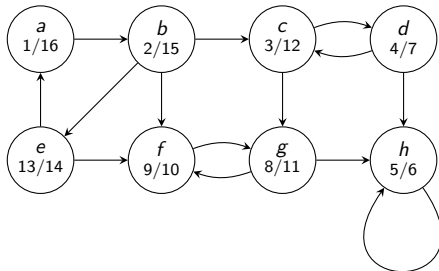
$a \rightarrow b$, $b \rightarrow c$, $b \rightarrow e$, $b \rightarrow f$, $c \rightarrow d$, $c \rightarrow g$, $d \rightarrow c$, $d \rightarrow h$, $e \rightarrow a$,
 $e \rightarrow f$, $f \rightarrow g$, $g \rightarrow f$, $g \rightarrow h$, $h \rightarrow h$.

El grafo de partida

Las listas de adyacencia

En el orden en que se leen las aristas: $a: [b]; \quad b: [c, e, f]; \quad c: [d, g]; \quad d: [c, h]; \quad e: [a, f]; \quad f: [g]; \quad g: [f, h]; \quad h: [h]$.

Primer paso: la profundidad sobre G



Cómo se leyeron los tiempos

La búsqueda arranca en a y no le hace falta arrancar de nuevo: desde a se alcanza todo. Baja por a, b, c, d, h ; vuelve a c y baja por g y f ; vuelve a b y baja por e .

Primer paso: la profundidad sobre G

El anidamiento de paréntesis

(a (b (c (d (h)) (g (f))) (e)))

Las aristas que no son del árbol

Arista	Tipo	Por qué
$b \rightarrow f$	avance	f negro y $b.d < f.d$
$d \rightarrow c$	retroceso	c gris
$e \rightarrow a$	retroceso	a gris
$e \rightarrow f$	cruzada	f negro y $f.d < e.d$
$f \rightarrow g$	retroceso	g gris
$g \rightarrow h$	cruzada	h negro y $h.d < g.d$
$h \rightarrow h$	retroceso	lazo sobre sí mismo

Segundo paso: el orden por f decreciente

Los ocho vértices ordenados

Vértice	a	b	e	c	g	f	d	h
f	16	15	14	12	11	10	7	6

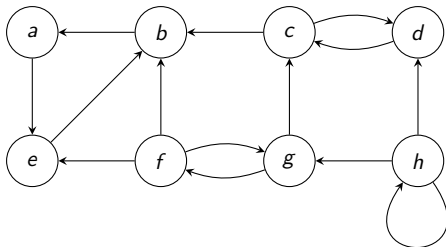
ord

$$ord = [a, b, e, c, g, f, d, h]$$

Qué ordena ord

A los componentes. Leído sobre G^{SCC} , ord da $\{a, b, e\}$, $\{c, d\}$, $\{f, g\}$, $\{h\}$, que es un orden topológico de ese DAG. Sobre G la lista no ordena nada, y la arista $e \rightarrow a$ lo muestra: va hacia atrás.

Tercer paso: el transpuesto

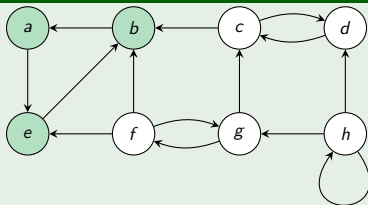


Las listas de G^T

a : [e]; b : [a]; c : [b, d]; d : [c]; e : [b]; f : [b, e, g]; g : [c, f]; h : [d, g, h].

Cuarto paso: *Asignar* sobre G^T , primeros dos componentes

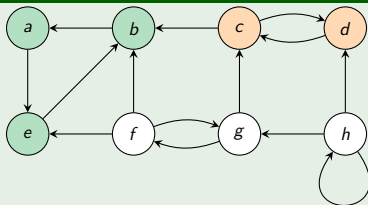
Asignar(a, a)



De a se va a e , de e a b , y de b se vuelve a a , que ya tiene componente. Quedan pintados $\{a, b, e\}$. En G^T de este bloque no sale ninguna arista hacia afuera.

Cuarto paso: *Asignar* sobre G^T , primeros dos componentes

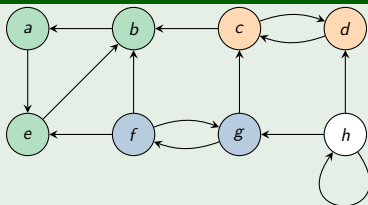
Asignar(c, c)



b y e ya están asignados y el paso 3 de *ord* los salta. El siguiente sin asignar es c . De c se va a b , que ya tiene componente, y a d ; de d se vuelve a c . Quedan $\{c, d\}$.

Cuarto paso: los otros dos componentes

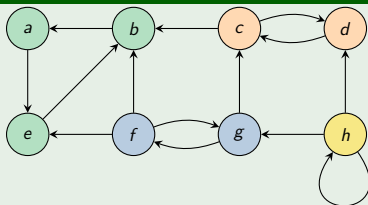
Asignar(g, g)



De g se va a c , asignado, y a f ; de f se va a b y e , asignados, y a g , asignado. Quedan $\{f, g\}$.

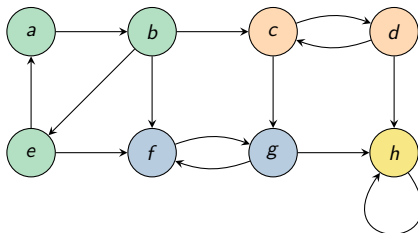
Cuarto paso: los otros dos componentes

Asignar(h, h)



f y d ya están asignados. Queda h : de él salen aristas a d , a g y a sí mismo, todas asignadas. El componente es $\{h\}$, él solo.

El resultado

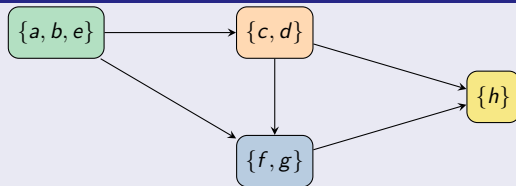


Los cuatro componentes, en el orden en que salieron

$\{a, b, e\}$, $\{c, d\}$, $\{f, g\}$, $\{h\}$.

El resultado

Y el grafo de componentes que queda



Una propina

El orden en que Kosaraju entrega los componentes es un orden topológico de G^{SCC} . Sale gratis: el paso 3 los recorre por f decreciente y las aristas de G^{SCC} van siempre de mayor a menor f .

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes**
- 10 Ejercicios
- 11 Referencias

Errores comunes

1. Contestar con los componentes del no dirigido

Ignorar las direcciones y correr el algoritmo de componentes conexos responde otra pregunta. Sobre el grafo de la sesión, sin direcciones, la respuesta es un solo componente; con direcciones son cuatro. El primero es el número de componentes débilmente conexos, y no dice nada sobre los fuertes.

2. Recorrer G^T en cualquier orden

Arrancando la segunda pasada en h en vez de en a , $Asignar(h, h)$ sobre G^T llega a d, c, b, a, e, g y f : los ocho vértices en un solo componente. El orden por f decreciente no es un detalle de implementación, es lo que hace correcto al algoritmo.

Errores comunes

3. Dejar por fuera los vértices aislados

Un vértice sin ninguna arista es un componente conexo de tamaño uno, y un vértice del que no se puede volver es un componente fuertemente conexo de tamaño uno. Recorrer la lista de aristas en vez de recorrer $0 \dots n - 1$ los deja sin contar, y la respuesta sale corta.

4. Confundir débil con fuerte

Todo grafo fuertemente conexo es débilmente conexo; al revés, no. Una fila $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ es débilmente conexa y tiene cuatro componentes fuertemente conexos. Y “el grafo tiene un solo componente débil” nunca implica “es fuertemente conexo”.

Errores comunes

5. Transponer a medias

G^T es una estructura nueva. Invertir las aristas encima de G deja a la mitad del recorrido con las listas viejas y a la otra mitad con las nuevas, y el resultado no es ni lo uno ni lo otro. El paso 2 construye un arreglo aparte.

6. Marcar el vértice tarde

En *CCDFSAux* el orden de los pasos importa: primero marcar, después mirar los adyacentes. Al revés, el primer ciclo del grafo hace que la recursión entre otra vez al vértice que ya estaba en curso y no termine nunca.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios**
- 11 Referencias

Ejercicios

1. Componentes conexos a mano

Sea G el grafo no dirigido con $V = \{0, 1, \dots, 8\}$ y aristas $0-3$, $3-6$, $6-0$, $1-4$, $4-7$, $2-5$, $5-8$, $8-2$. Dibuje el grafo, liste sus componentes conexos y diga en qué orden los encuentra *ComponentesConexosDFS* si el ciclo del paso 1 recorre los vértices de 0 a 8.

2. Profundidad y amplitud sobre el mismo grafo

Sobre el grafo del ejercicio 1, escriba el orden en que *CCDFSAux* visita los vértices del componente de 0 y el orden en que los visita *CCBFSAux*. Diga qué cambia y qué no entre las dos listas, y por qué el reparto en componentes sale igual.

Ejercicios

3. Puntos de articulación y puentes

Sea H el grafo no dirigido con $V = \{0, 1, \dots, 6\}$ y aristas $0-1, 1-2, 2-0, 2-3, 3-4, 4-5, 5-6, 6-4$. Liste todos los puntos de articulación y todos los puentes. Para cada uno, diga cuántos componentes conexos quedan al removerlo y cuáles son. Para cada vértice y cada arista que no esté en su lista, diga por qué no lo está.

4. Débil, fuerte y componentes

Sea D el grafo dirigido con $V = \{a, b, c, d, e, f\}$ y aristas $a \rightarrow b, b \rightarrow c, c \rightarrow a, c \rightarrow d, d \rightarrow e, e \rightarrow d, f \rightarrow e$. Conteste, justificando cada respuesta: ¿es débilmente conexo?, ¿es fuertemente conexo?, ¿cuáles son sus componentes fuertemente conexos?, ¿cómo queda D^{SCC} ?

Ejercicios

5. Kosaraju a mano

Sea K el grafo dirigido con $V = \{1, 2, 3, 4, 5, 6\}$ y aristas $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$, $3 \rightarrow 4$, $4 \rightarrow 5$, $5 \rightarrow 6$, $6 \rightarrow 4$. Ejecute el algoritmo de Kosaraju: calcule $v.d$ y $v.f$ de la primera profundidad arrancando en 1, escriba ord , dibuje K^T y muestre qué componente pinta cada llamada a *Asignar*. Verifique al final que para cada arista de K^{SCC} se cumple $f(C) > f(C')$.

6. Un puente no está en ningún ciclo

Demuestre que si (u, v) es un puente de un grafo no dirigido conexo, entonces (u, v) no pertenece a ningún ciclo de ese grafo. Escriba la demostración en cuatro partes: teorema, estrategia, desarrollo y conclusión.

Ejercicios

7. El extremo contrario

Dé un grafo dirigido de cuatro vértices que sea débilmente conexo y tenga cuatro componentes fuertemente conexos. ¿Cuál es el mínimo número de aristas que necesita? Justifique por qué con menos no alcanza. Después, agregue una sola arista a su grafo de modo que quede con un único componente fuertemente conexo, y explique cuál eligió.

8. Qué pasa si se salta el transpuesto

Sobre el grafo de la sesión, ejecute el paso 3 de Kosaraju recorriendo *ord* pero llamando a *Asignar* sobre G en vez de sobre G^T . Escriba los “componentes” que salen y señale en cuál de los dos argumentos de la demostración se apoyaba el transpuesto.

Lo que sigue

Los mismos componentes, con un solo recorrido

Kosaraju necesita dos búsquedas en profundidad y una copia del grafo al revés. Los algoritmos de Tarjan y de Gabow encuentran los mismos componentes con una sola búsqueda y sin construir G^T : mientras bajan, guardan los vértices en una pila y llevan en cada uno un segundo número, $v.low$, que dice cuál es el tiempo de descubrimiento más pequeño al que se puede llegar desde su subarbol por una arista de retroceso.

Para investigar

Qué es $v.low$, cómo se calcula al volver de cada llamada recursiva y por qué la condición $v.low = v.d$ marca la raíz de un componente. Con ese mismo valor se resuelven después los puentes y los puntos de articulación, que hoy quedaron solo como definición: están en el Problema 22-2 de CLRS.

Contenido

- 1 Antes de empezar
- 2 Recuento
- 3 Conectividad
- 4 Componentes conexos con los recorridos
- 5 Puntos de articulación y puentes
- 6 Componentes fuertemente conexos
- 7 El algoritmo de Kosaraju
- 8 Paso a paso
- 9 Errores comunes
- 10 Ejercicios
- 11 Referencias**

Referencias

-  Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. Sección 22.2 y Teorema 22.5 (búsqueda en amplitud); Sección 22.3 y Teorema 22.7 (teorema del paréntesis y clasificación de aristas); Sección 22.5 (componentes fuertemente conexos, grafo transpuesto y grafo de componentes); Problema 22-2 (puntos de articulación, puentes y componentes biconexos).
-  Kosaraju, S. R. Algoritmo no publicado, 1978. La primera descripción impresa aparece en Sharir, M. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, 1981.
-  van Steen, M. *Graph Theory and Complex Networks: An Introduction*. 2010. Conectividad, componentes y el papel de los vértices de corte en redes reales.

Referencias



Halim, S., Halim, F., Effendy, S. *Competitive Programming 4*. Lulu, 2020. Capítulo 4, recorridos en grafos: componentes conexos, puntos de articulación y puentes, y componentes fuertemente conexos.